

Proyecto Analítica de Textos - Etapa 2

Documentación de la API

Integrantes:

- Adriana Sofia Rozo Cepeda
- Tomas Alberto Rodriguez Peña
- Andrés Caballero

Fecha: 29 de marzo de 2025

Curso: Inteligencia de Negocios

Índice general

1. Contexto del Proyecto	1
1.1. Contexto y Justificación	1
1.2. Motivación	1
1.3. Objetivos	1
1.4. Alcance	1
2. Proceso de Automatización del Modelo	2
2.1. Creación del Pipeline	2
2.1.1. Preprocesamiento de Datos	2
2.1.2. Creación y Entrenamiento del Modelo	2
2.1.3. Persistencia del Modelo	3
2.2. Implementación de la API	3
2.3. Definiciones de Re-entrenamiento	5
2.3.1. Reentrenamiento completo del modelo	5
2.3.2. Aprendizaje incremental	6
2.3.3. Enseñanza activa	6
2.3.4. Conclusión: Método Implementado	6
3. Desarrollo de la Aplicación y Justificación	7
3.1. Descripción del Usuario/Rol	7
3.2. Conexión con el Proceso de Negocio	7
3.3. Importancia para los Usuarios Finales	7
4. Recursos e Integración	8
5. Resultados	9
5.1. Video de Resultados	9
5.2. Impacto en Colombia	9
6. Trabajo en Equipo	10
6.1. Roles y Tareas	10
6.2. Reuniones de Grupo	11

Capítulo 1

Contexto del Proyecto

1.1. Contexto y Justificación

La proliferación de noticias falsas sobre política, tanto a nivel local como internacional, afecta la confianza en las instituciones democráticas y debilita la estabilidad social. Para afrontar esta problemática, se decidió aplicar técnicas de aprendizaje automático al análisis de noticias, permitiendo procesar grandes volúmenes de texto y detectar rápidamente contenido engañoso.

1.2. Motivación

La necesidad de un sistema automatizado surge al manejar constantemente noticias generadas en múltiples plataformas. Un modelo de aprendizaje automático puede mejorar sustancialmente la precisión, minimizando los riesgos de desinformación y polarización social.

1.3. Objetivos

- **Análisis automatizado:** Implementar un *pipeline* que realice preprocesamiento y clasificación de noticias.
- **Actualización continua:** Reentrenar el modelo con nuevos datos para mantener una predicción precisa.
- **API accesible:** Exponer las funcionalidades del modelo mediante *FastAPI*, facilitando su integración en diversos entornos.

1.4. Alcance

El proyecto comprende el desarrollo de un prototipo para clasificar noticias (reales/falsas) con un enfoque orientado al dominio político, integrando un flujo completo de datos y un servicio de inferencia y reentrenamiento desplegado como API.

Capítulo 2

Proceso de Automatización del Modelo

2.1. Creación del Pipeline

El proceso de automatización del *pipeline* desarrollado para la analítica de textos consta de varias etapas clave: la preparación de datos, la construcción del modelo y la persistencia del modelo entrenado.

2.1.1. Preprocesamiento de Datos

Para asegurar la calidad de los datos, se aplican diversas transformaciones. Entre ellas:

- **Tokenización:** División del texto en palabras individuales.
- **Eliminación de puntuación:** Filtrado de caracteres no alfabéticos.
- **Conversión a minúsculas:** Uniformiza el texto para evitar duplicaciones por cambio de mayúsculas.
- **Reemplazo de números:** Conversión de números a su representación textual en español.
- **Eliminación de caracteres no ASCII:** Limpieza de caracteres incompatibles.
- **Eliminación de stopwords:** Supresión de palabras irrelevantes (ej. preposiciones, artículos).
- **Stemming:** Reducción de palabras a su raíz con *Snowball Stemmer* en español.

Estas operaciones se aplican a las columnas `Titulo` y `Descripcion` para crear la columna combinada `Texto_Procesado`, usada finalmente por el modelo.

2.1.2. Creación y Entrenamiento del Modelo

El modelo se compone de un *TfidfVectorizer* y un clasificador *Random Forest*. El flujo de entrenamiento incluye:

1. **Separación de datos:** 70 % para entrenamiento y 30 % para validación.
2. **Vectorización del texto:** Uso de `TfidfVectorizer` para obtener representaciones numéricas de las noticias.
3. **Entrenamiento del modelo:** Modelo de *Random Forest* con 300 árboles y parámetros ajustados (`min_samples_split=10`, sin límite de profundidad).

Las métricas principales (precisión, *recall*, F1-score) y la matriz de confusión evidencian un comportamiento robusto, con precisión global alrededor del 94 %.

2.1.3. Persistencia del Modelo

Para su reutilización y despliegue, el modelo se guarda en un archivo `.joblib`, de modo que cualquier aplicación pueda cargarlo para realizar predicciones sin reentrenamiento inmediato.

2.2. Implementación de la API

Se desarrolló una API con **FastAPI**, permitiendo:

- `/predict/`: Recibe una lista de noticias (título y descripción) y devuelve la clasificación y la probabilidad de cada predicción.
- `/predict_csv/`: Procesa un archivo CSV sin columna de etiquetas, ofreciendo predicciones masivas.
- `/retrain/`: Permite reentrenar el modelo desde cero con nuevos datos, asegurando la adaptación continua.

Estos *endpoints* usan *schemas* Pydantic para validar la estructura de entrada y salida, asegurando la coherencia de los datos.

A continuación se muestran los fragmentos de código correspondientes a cada *endpoint*:

`/predict/`

```
1 @app.post("/predict/")
2 def predict_news(news_list: List[NewsInput]):
3     modelo = get_model()
4     df = pd.DataFrame([item.dict() for item in news_list])
5     predicciones = modelo.predict(df)
6     probabilidades = modelo.predict_proba(df).tolist()
7     resultados = [{
8         "Titulo": row.Titulo,
9         "Descripcion": row.Descripcion,
10        "Prediccion": "Real" if pred == 1 else "Falsa",
```

```
11     "Probabilidad": prob
12 } for row, pred, prob in zip(news_list, predicciones, probabilidades)]
13 return resultados
```

/predict_csv/

```
1 @app.post("/predict_csv/")
2 async def predict_csv(file: UploadFile = File(...)):
3     temp_path = "app/data/temp_predict.csv"
4     with open(temp_path, "wb") as f:
5         f.write(file.file.read())
6     df = pd.read_csv(temp_path, sep=";")
7     predicciones = modelo.predict(df)
8     probabilidades = modelo.predict_proba(df).tolist()
9     resultados = [{
10         "Titulo": row.Titulo,
11         "Descripcion": row.Descripcion,
12         "Prediccion": "Real" if pred == 1 else "Falsa",
13         "Probabilidad": {
14             "Real": round(prob[1], 4),
15             "Falsa": round(prob[0], 4)
16         }
17     } for row, pred, prob in zip(df.itertuples(), predicciones, probabilidades)]
18     return {"predicciones": resultados}
```

/retrain/

```
1 @app.post("/retrain/")
2 async def retrain_model(file: UploadFile = File(...)):
3     temp_path = "app/data/temp_retrain.csv"
4     with open(temp_path, "wb") as f:
5         f.write(file.file.read())
6     df = pd.read_csv(temp_path, sep=";")
7     df['Texto_Procesado'] = transfor_data_local(df)
8     text_transformer = TfidfVectorizer(max_features=10000, ngram_range=(1, 2))
9     results, nuevo_modelo = createModel(text_transformer, df)
10    joblib.dump(nuevo_modelo, MODEL_PATH)
11    global modelo
12    modelo = nuevo_modelo
13    return {
14        "message": "Modelo actualizado correctamente",
15        "metrics": results
16    }
```

Esta estructura de la API garantiza la disponibilidad del modelo y su capacidad para manejar grandes volúmenes de datos de manera eficiente. Su implementación permite integrar el modelo en aplicaciones externas, facilitando la automatización de la clasificación de noticias y reduciendo significativamente el tiempo de moderación.


Fake News Detector
 Detecta si una noticia es falsa o real utilizando IA.

Realizar Predicciones

Título de la noticia

La 'Ayusización' del BNC: Santiago Abascal instruye a los suyos a seguir la estela de la presidenta madrileña

Descripción de la noticia

Pablo Santiago Abascal planea vivir de las rentas de la arrolladora victoria de Isabel Díaz Pablo todo 10 dure la Sin embargo, diversas voces del BNG creen su tirón no es al líder del BNG.

Predicir

✓ Predicción: **Falsa**
 ⚠ Probabilidad: 5.00%

Predicción Masiva desde CSV

Seleccionar archivo CSV (.csv)

Seleccionar archivo sample_news.csv

Predicir desde CSV

✓ Resultados:

Título	Descripción	Predicción	Probabilidad Real (%)	Probabilidad Falsa (%)
revel gobiern negoci lib mirel cambi otorg perdon javi duart	mexic gran polem asombr caus red social notici lanz revist polit asegur lib doctor jos manuel mirel valverd numb encarcel	Real	62.22%	37.78%
pnv consol mayori pse salv papel pp sig marginal	nacional consig alcald bilbo donosti vitori social logr ret principal feud mientr popul vuelv perd represent vari local	Real	97.59%	2.41%

Reentrenar el Modelo

Seleccionar archivo CSV para reentrenamiento (.csv)

Seleccionar archivo fake_news_spanish.csv

Reentrenar

✓ ¡Reentrenamiento exitoso!
 ⚠ Precisión: 94.07%
 📊 Recall: 93.95%
 📊 F1 Score: 93.91%

Figura 2.1: Vista de la API entregada

2.3. Definiciones de Re-entrenamiento

Para asegurar que el modelo se mantenga actualizado con nuevos datos y continúe ofreciendo predicciones precisas, se consideraron tres enfoques:

2.3.1. Reentrenamiento completo del modelo

Implica reentrenar el modelo desde cero utilizando el conjunto completo de datos históricos más los datos nuevos. El modelo resultante reemplaza al anterior.

Ventaja: Integra toda la información disponible, minimizando sesgos acumulativos. **Desventaja:** Alto costo computacional si el volumen de datos crece rápidamente.

2.3.2. Aprendizaje incremental

Ajusta los pesos y parámetros del modelo actual con los datos nuevos, sin reentrenar con todo el histórico.

Ventaja: Más rápido y eficiente en términos computacionales. **Desventaja:** Puede acumular errores si los datos no representan adecuadamente la distribución general.

2.3.3. Enseñanza activa

Selecciona estratégicamente un subconjunto de datos más “informativos” para reentrenar, reduciendo el tiempo de entrenamiento.

Ventaja: Optimiza recursos al enfocarse en datos relevantes. **Desventaja:** Requiere una estrategia eficiente de selección, con riesgo de omitir ejemplos importantes.

2.3.4. Conclusión: Método Implementado

Dadas las necesidades del proyecto, se optó por **reentrenamiento completo**, garantizando la incorporación de toda la información disponible a costa de un mayor consumo computacional, pero con menor riesgo de sesgos o pérdida de información.

Capítulo 3

Desarrollo de la Aplicación y Justificación

3.1. Descripción del Usuario/Rol

La aplicación de detección de noticias falsas en política está diseñada para distintos perfiles:

- **Periodistas y verificadores:** Necesitan validar la veracidad de las noticias antes de publicar.
- **Entidades gubernamentales:** Requieren evaluar la calidad de la información pública y detectar campañas de desinformación.
- **Consumidores de noticias:** Buscan una fuente de información confiable, reduciendo el riesgo de contenido engañoso.

3.2. Conexión con el Proceso de Negocio

La solución respalda el **proceso de verificación y moderación de contenido**, contribuyendo a la identificación temprana de noticias falsas. Con ello, se reducen riesgos de desinformación a escala tanto en medios de comunicación como en redes oficiales.

3.3. Importancia para los Usuarios Finales

- Los periodistas mejoran su eficiencia al verificar más noticias en menos tiempo.
- Los reguladores gubernamentales pueden usar las salidas analíticas para diseñar estrategias que mitiguen la manipulación informativa.
- Los lectores obtienen mayor confianza y seguridad en los canales noticiosos.

Capítulo 4

Recursos e Integración

Recursos informáticos requeridos:

- **Entrenamiento y ejecución del modelo:** Se recomienda un servidor o entorno con capacidad de procesamiento (CPU multinúcleo, al menos 8 GB de RAM). Para volúmenes mayores, podría requerirse GPU.
- **Persistencia del modelo:** Espacio en disco para almacenar el archivo `.joblib`.
- **Despliegue de la aplicación:** Un servidor de aplicaciones como *uvicorn/gunicorn* para FastAPI, con conectividad. Opcionalmente, contenedores Docker para simplificar el proceso.

Integración con la organización y disposición al usuario final:

- **Conexión con procesos del negocio:** Moderar el flujo de noticias antes de publicar o incluir la API en un sistema de monitoreo de redes sociales.
- **Disponibilidad al usuario final:** Expongamos *endpoints* seguros en la intranet o internet para que diferentes usuarios puedan enviar titulares y descripciones, obteniendo la clasificación.

Riesgos para el usuario final:

- **Falsos positivos/negativos:** El modelo no es infalible.
- **Sesgo en los datos:** Puede reflejarse en las predicciones si no se considera un conjunto de entrenamiento balanceado.
- **Dependencia de la herramienta:** Requiere monitoreo y revisiones periódicas para evitar degradación de la precisión.

Capítulo 5

Resultados

5.1. Video de Resultados

Enlace de la demostración en video: <https://www.youtube.com/watch?v=JJ9K19zuWLA>

5.2. Impacto en Colombia

La aplicación contribuye a mitigar la desinformación en el ámbito político. Ayuda a mantener la estabilidad y reduce la polarización, fortaleciendo la transparencia institucional y la credibilidad de los medios ante la audiencia.

Capítulo 6

Trabajo en Equipo

6.1. Roles y Tareas

Adriana Sofia Rozo Cepeda (Líder de Proyecto)

- **Rol:** Gestión general del proyecto (fechas, asignaciones, entregables).
- **Tareas Principales:** Coordinación de la documentación y supervisión de preprocesamientos.
- **Tiempo dedicado:** 15 horas.

Tomas Alberto Rodriguez Peña (Ingeniero de Datos)

- **Rol:** Construcción del modelo analítico y pipeline de datos.
- **Tareas Principales:** Implementación de transformaciones de texto, entrenamiento del Random Forest y validación de métricas.
- **Tiempo dedicado:** 15 horas.

Andrés Caballero (Ingeniero de Software)

- **Rol:** Desarrollo de la API en FastAPI y creación del video de resultados.
- **Tareas Principales:** Diseño de endpoints (`/predict/`, `/predict_csv/`, `/retrain/`), validaciones con *Pydantic* y edición del video.
- **Tiempo dedicado:** 15 horas.

Distribución de 100 puntos

Cada integrante recibió 33.3 puntos, reconociendo la contribución equitativa.

6.2. Reuniones de Grupo

Reunión de Lanzamiento (15/03/2025):

- Definición de roles, plan de trabajo y hito final.

Reunión de Seguimiento (20/03/2025):

- Validación del pipeline y prototipo API.

Reunión de Finalización (28/03/2025):

- Integración de secciones, preparación de video y entrega final.

Aspectos a mejorar:

- Explorar aprendizaje incremental para el *retrain*.
- Ampliar la variedad de datos para reducir sesgos.
- Documentar en mayor detalle los ajustes de hiperparámetros.

Bibliografía

- [1] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. Recuperado de <https://www.springer.com/gp/book/9780387310732>
- [2] Ramos Fernández, J. (2018). *Aprendizaje automático para flujos de datos*. Universidad Politécnica de Madrid. Recuperado de https://cig.fi.upm.es/wp-content/uploads/2024/01/TFM_JAVIER_RAMOS_FERNANDEZ.pdf
- [3] Settles, B. (2012). *Active Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning, 6(1), 1-114. Recuperado de https://www.cs.cmu.edu/~bsettles/active_learning.pdf